

Python with MySQL

Complete CRUD Demonstration Guide

Installation • Database Setup • Static & Dynamic Insert • Read • Update • Delete

Learning outcome

By the end of this lesson, learners can connect Python to MySQL and perform all CRUD operations safely using user input and parameterized SQL.

Prepared by Thanesh

Prepared for classroom demonstration and hands-on practice

Examples use: database company_db and table employees.

1. Course Roadmap

Step	Topic	Practical outcome
1	Install software and package	Prepare Python, MySQL and connector
2	Create database and table	Build company_db.employees
3	Test connection	Confirm Python can access MySQL
4	Insert	Static and user-driven records
5	Read	Display all records and search by ID
6	Update	Modify a selected employee
7	Delete	Remove a selected employee safely

CRUD is the standard name for the four basic database operations:

Operation	SQL	Meaning
Create	INSERT	Add a new record
Read	SELECT	Retrieve existing records
Update	UPDATE	Modify an existing record
Delete	DELETE	Remove an existing record

2. Required Software and Package Installation

- Python 3.x - runs the Python programs.
- MySQL Server - stores the database and employee records.
- MySQL Workbench (recommended) - graphical tool to run SQL and inspect data.
- VS Code / PyCharm / IDLE - editor used to write and run Python code.
- mysql-connector-python - package that lets Python communicate with MySQL.

Open Command Prompt or the terminal and run:

COMMAND PROMPT

```
python --version
pip --version
pip install mysql-connector-python
```

If pip is not recognized, use:

COMMAND PROMPT

```
python -m pip install mysql-connector-python
```

Verify the installation:

COMMAND PROMPT

```
python -c "import mysql.connector; print('Connector installed successfully')"
```

Important

Start the MySQL Server before running the Python programs. Keep your database username and password ready.

Recommended project files

File	Purpose
connection_test.py	Test MySQL connectivity
insert_static.py	Insert fixed values
insert_dynamic.py	Insert user values
read_employees.py	Display records
update_employee.py	Update by ID
delete_employee.py	Delete by ID

3. Database and Table Setup

Open MySQL Workbench (or the MySQL command line), connect as root, and execute the complete script below.

MYSQL

```
CREATE DATABASE IF NOT EXISTS company_db;

USE company_db;

CREATE TABLE IF NOT EXISTS employees (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(100) NOT NULL UNIQUE,
  salary DECIMAL(10, 2) NOT NULL
);

DESCRIBE employees;
```

Column	Type	Purpose
id	INT, Primary Key, Auto Increment	Unique ID generated by MySQL
name	VARCHAR(100), NOT NULL	Employee's name
email	VARCHAR(100), UNIQUE	Employee's unique email
salary	DECIMAL(10,2), NOT NULL	Salary with two decimal places

Why AUTO_INCREMENT?

The learner does not enter the ID. MySQL automatically generates 1, 2, 3, and so on.

4. Test the Python-MySQL Connection

PYTHON

```
import mysql.connector

try:
    connection = mysql.connector.connect(
        host="localhost",
        user="root",
        password="root",
        database="company_db"
    )

    if connection.is_connected():
        print("Successfully connected to MySQL")

except mysql.connector.Error as error:
    print("Connection failed:", error)

finally:
    if "connection" in locals() and connection.is_connected():
        connection.close()
        print("Connection closed")
```

Connection properties

Property	Example	Meaning
host	localhost	MySQL runs on this computer
user	root	MySQL username
password	root	MySQL password
database	company_db	Database used by the program

Change the password

If your MySQL password is not root, replace password="root" with your actual password.

5. Static Insert

A static insert uses values already written inside the program. It is useful for understanding the basic INSERT flow.

PYTHON

```
import mysql.connector

connection = mysql.connector.connect(
    host="localhost",
    user="root",
    password="root",
    database="company_db"
)

cursor = connection.cursor()

sql = """
INSERT INTO employees (name, email, salary)
VALUES (%s, %s, %s)
"""

employee = ("Rahul", "rahul@gmail.com", 45000)

cursor.execute(sql, employee)
connection.commit()

print("Employee inserted successfully")
print("Employee ID:", cursor.lastrowid)

cursor.close()
connection.close()
```

Code explanation

cursor() sends SQL to MySQL. execute() runs the query. commit() permanently saves the change. lastrowid returns the ID generated for the inserted row.

Expected output

OUTPUT

```
Employee inserted successfully
Employee ID: 1
```

6. Dynamic Insert Using input()

A dynamic insert accepts values from the user while the program is running.

PYTHON

```
import mysql.connector

connection = mysql.connector.connect(
    host="localhost",
    user="root",
    password="root",
    database="company_db"
)

cursor = connection.cursor()

name = input("Enter employee name: ").strip()
email = input("Enter employee email: ").strip()
salary = float(input("Enter employee salary: "))

sql = """
INSERT INTO employees (name, email, salary)
VALUES (%s, %s, %s)
"""

values = (name, email, salary)
cursor.execute(sql, values)
connection.commit()

print("Employee inserted successfully")
print("Employee ID:", cursor.lastrowid)

cursor.close()
connection.close()
```

Sample execution

OUTPUT

```
Enter employee name: Priya
Enter employee email: priya@gmail.com
Enter employee salary: 55000
Employee inserted successfully
Employee ID: 2
```

Security best practice

Use %s placeholders and pass values separately. Never build SQL by joining raw user input because that can cause SQL injection.

7. Read Employee Records

The SELECT command reads data. A dictionary cursor lets us access columns using names such as `employee['name']`.

A. Display all employees

PYTHON

```
import mysql.connector

connection = mysql.connector.connect(
    host="localhost",
    user="root",
    password="root",
    database="company_db"
)

cursor = connection.cursor(dictionary=True)
cursor.execute("SELECT id, name, email, salary FROM employees")
employees = cursor.fetchall()

if employees:
    print("\nEmployee Details")
    print("-" * 65)
    for employee in employees:
        print(
            employee["id"],
            employee["name"],
            employee["email"],
            employee["salary"]
        )
else:
    print("No employees found")

cursor.close()
connection.close()
```

B. Search dynamically by employee ID

PYTHON

```
import mysql.connector

connection = mysql.connector.connect(
    host="localhost",
    user="root",
    password="root",
    database="company_db"
)

cursor = connection.cursor(dictionary=True)

employee_id = int(input("Enter employee ID to search: "))
sql = "SELECT id, name, email, salary FROM employees WHERE id = %s"

cursor.execute(sql, (employee_id,))
employee = cursor.fetchone()

if employee:
    print("ID:", employee["id"])
    print("Name:", employee["name"])
    print("Email:", employee["email"])
    print("Salary:", employee["salary"])
else:
    print("Employee ID not found")

cursor.close()
connection.close()
```

fetchall() vs fetchone()
fetchall() returns all matching rows. fetchone() returns one matching row.

8. Dynamic Update

This program uses the employee ID to select a record, then replaces its name, email, and salary with values entered by the user.

PYTHON

```
import mysql.connector

connection = mysql.connector.connect(
    host="localhost",
    user="root",
    password="root",
    database="company_db"
)

cursor = connection.cursor()

employee_id = int(input("Enter employee ID to update: "))
name = input("Enter new employee name: ").strip()
email = input("Enter new employee email: ").strip()
salary = float(input("Enter new employee salary: "))

sql = """
UPDATE employees
SET name = %s, email = %s, salary = %s
WHERE id = %s
"""

values = (name, email, salary, employee_id)
cursor.execute(sql, values)
connection.commit()

if cursor.rowcount > 0:
    print("Employee updated successfully")
else:
    print("Employee ID not found or values were unchanged")

cursor.close()
connection.close()
```

Sample execution

OUTPUT

```
Enter employee ID to update: 2
Enter new employee name: Priya Sharma
Enter new employee email: priya.sharma@gmail.com
Enter new employee salary: 60000
Employee updated successfully
```

Why WHERE is essential

Without WHERE id = %s, the UPDATE statement may modify every employee in the table.

9. Dynamic Delete

This program accepts an employee ID, asks for confirmation, and deletes only that selected employee.

PYTHON

```
import mysql.connector

connection = mysql.connector.connect(
    host="localhost",
    user="root",
    password="root",
    database="company_db"
)

cursor = connection.cursor()

employee_id = int(input("Enter employee ID to delete: "))
confirmation = input("Type YES to confirm deletion: ")

if confirmation.upper() == "YES":
    sql = "DELETE FROM employees WHERE id = %s"
    cursor.execute(sql, (employee_id,))
    connection.commit()

    if cursor.rowcount > 0:
        print("Employee deleted successfully")
    else:
        print("Employee ID not found")
else:
    print("Deletion cancelled")

cursor.close()
connection.close()
```

Why (employee_id,) has a comma

Python treats (employee_id,) as a one-item tuple. Without the comma, it is only a value inside parentheses.

Why WHERE is essential

Without WHERE id = %s, DELETE can remove every row from the table.

10. Professional Error-Handling Pattern

For classroom basics, the earlier programs are easier. In real applications, use try-except-finally so errors are handled and resources are always closed.

PYTHON

```
import mysql.connector

connection = None
cursor = None

try:
    connection = mysql.connector.connect(
        host="localhost",
        user="root",
        password="root",
        database="company_db"
    )

    cursor = connection.cursor()

    name = input("Enter employee name: ").strip()
    email = input("Enter employee email: ").strip()
    salary = float(input("Enter employee salary: "))

    if not name or not email or salary < 0:
        raise ValueError("Enter valid employee details")

    sql = """
    INSERT INTO employees (name, email, salary)
    VALUES (%s, %s, %s)
    """

    cursor.execute(sql, (name, email, salary))
    connection.commit()
    print("Employee inserted successfully")

except ValueError as error:
    print("Invalid input:", error)

except mysql.connector.Error as error:
    if connection:
        connection.rollback()
    print("Database error:", error)

finally:
    if cursor:
        cursor.close()
    if connection and connection.is_connected():
        connection.close()
```

rollback()

If a database error occurs, `rollback()` cancels the uncommitted transaction and keeps the data consistent.

11. Key Concepts and Common Errors

Item	Purpose
connect()	Opens the connection to MySQL
cursor()	Creates an object for executing SQL
execute()	Runs one SQL statement
commit()	Permanently saves INSERT, UPDATE, or DELETE
rollback()	Cancels an uncommitted transaction
fetchone()	Reads one result row
fetchall()	Reads all result rows
rowcount	Number of affected rows
lastrowid	Auto-generated ID of the inserted row
close()	Releases cursor and connection resources

Common problems

Error	Likely reason / solution
ModuleNotFoundError	Install: <code>python -m pip install mysql-connector-python</code>
Access denied for user	Check MySQL username and password
Unknown database	Create <code>company_db</code> before running Python
Table doesn't exist	Create <code>employees</code> table and select <code>company_db</code>
Duplicate entry for email	The email column is UNIQUE; enter another email
Can't connect to MySQL server	Start the MySQL service and verify host/port
ValueError	Enter a numeric ID and salary where requested

12. Classroom Demonstration Plan

Sequence	Trainer action	Learner observation
1	Run CREATE DATABASE and CREATE TABLE	Schema is ready
2	Run connection_test.py	Connection succeeds
3	Run static insert	A fixed employee is inserted
4	Run dynamic insert twice	Different user values are saved
5	Run read-all program	All records are displayed
6	Search by an existing and invalid ID	fetchone() and not-found handling
7	Update one employee	Only the selected row changes
8	Delete one employee	Confirmation and rowcount are demonstrated
9	Run SELECT * in Workbench	Final database state is verified

Final verification SQL

MYSQL

```
USE company_db;
SELECT * FROM employees ORDER BY id;
```

Best practices recap

Use parameterized queries. Validate input. Always include WHERE for targeted update/delete. Commit data changes. Close the cursor and connection. Do not hard-code production passwords; use environment variables.

End of Guide

You now have a complete foundation for integrating Python applications with MySQL using CRUD operations.